

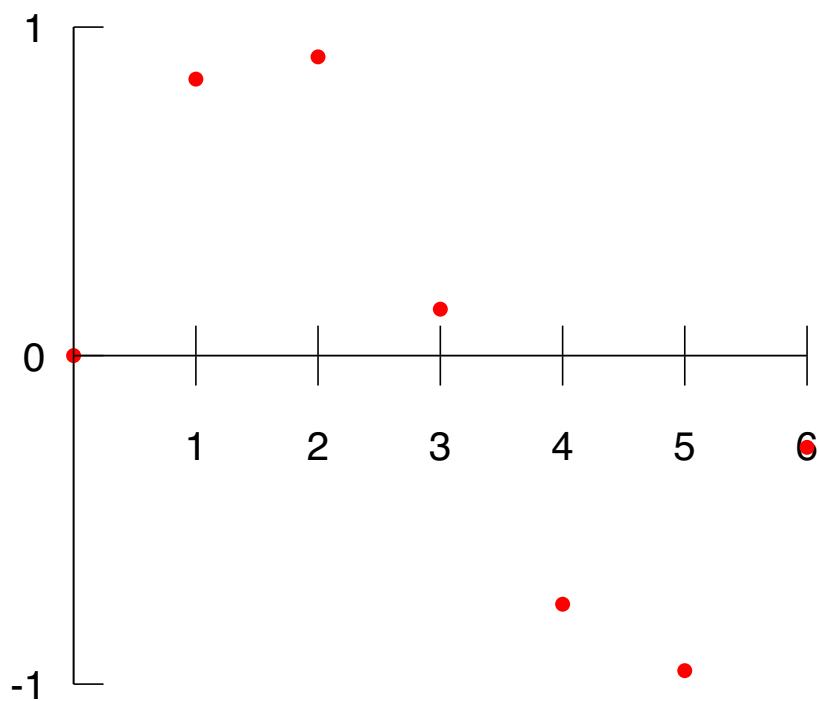
Interp

插值

引入

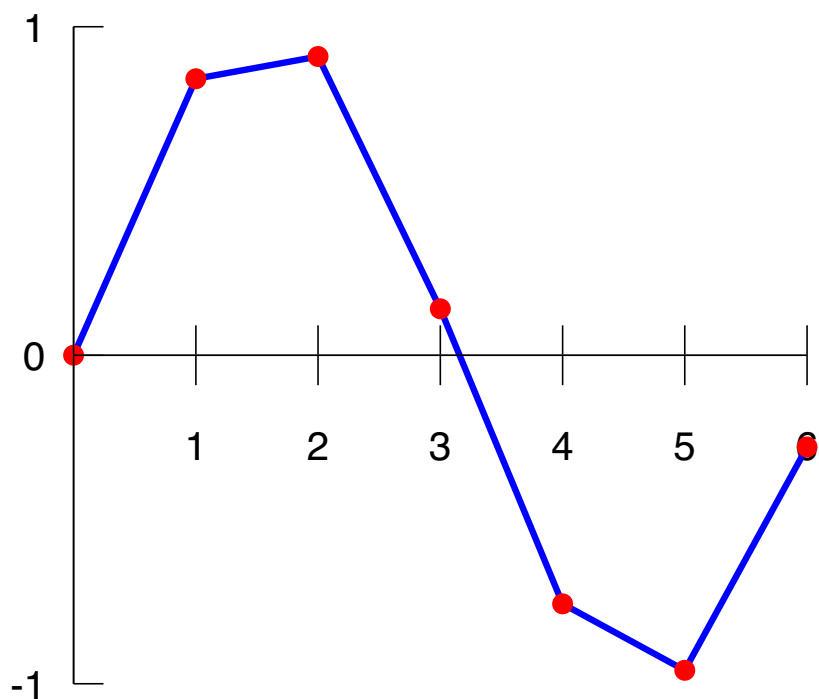
插值是一种通过已知的、离散的数据点推算一定范围内的新数据点的方法。插值法常用于函数拟合中。

例如对数据点：



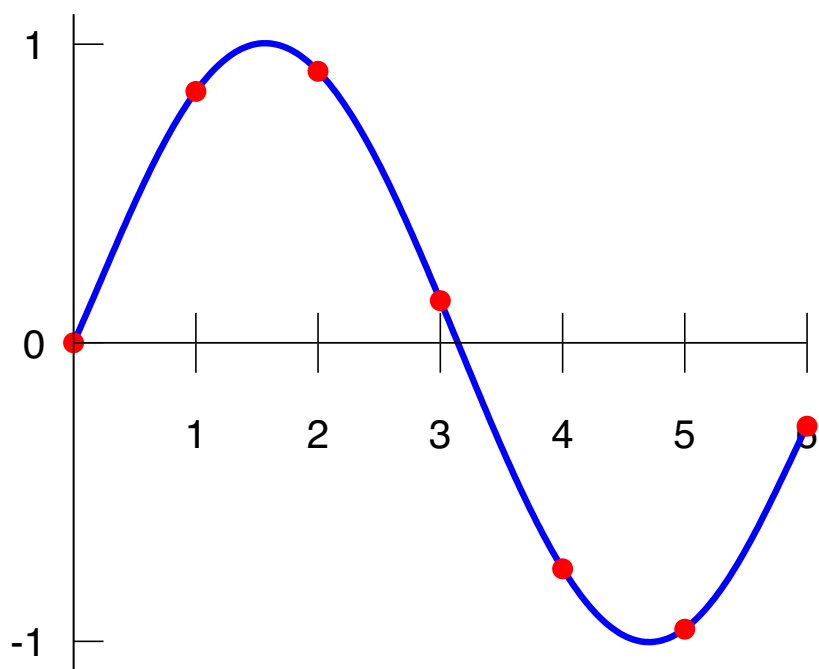
其中 未知，插值法可以通过按一定形式拟合 的方式估算未知的数据点。

例如，我们可以用分段线性函数拟合：



这种插值方式叫做 [线性插值](#)。

我们也可以用多项式拟合：



这种插值方式叫做 [多项式插值](#)。

多项式插值的一般形式如下：

▼ 多项式插值

对已知的 $n+1$ 个点，求 n 阶多项式 满足

下面介绍多项式插值中的两种方式：Lagrange 插值法与 Newton 插值法。不难证明这两种方法得到的结果是相等的。

Lagrange 插值法

由于要求构造一个函数过点，首先设第 i 个点在 x 轴上的投影为 x_i 。

考虑构造 n 个函数，使得对于第 i 个函数，其图像过 (x_i, y_i) ，则可知题目所求的函数 $L(x)$ 。

那么可以设 $L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$ ，将点 (x_i, y_i) 代入可以知道，所以

那么我们就可以得出 Lagrange 插值的形式为：

朴素实现的时间复杂度为 $O(n^2)$ ，可以优化到 $O(n^2)$ ，参见 [多项式快速插值](#)。

▼ [Luogu P4781【模板】拉格朗日插值](#)

给出 n 个点对 (x_i, y_i) ，且 x_i 互不相同，求 $L(x)$ 。

- 题解
- 代码实现

横坐标是连续整数的 Lagrange 插值

如果已知点的横坐标是连续整数，我们可以做到 $O(n)$ 插值。

设要求 n 次多项式为 $L(x)$ ，我们已知 (i, y_i) ，考虑代入上面的插值公式：

后面的累乘可以分子分母分别考虑，不难得到分子为：

分母的累乘可以拆成两段阶乘来算：

于是横坐标为 x 的插值公式：

预处理 $n!$ 前后缀积、阶乘阶乘逆，然后代入这个式子，复杂度为 $O(n)$ 。

▼ 例题 [CF622F The Sum of the k-th Powers](#)

给出 n ，求 $\sum_{i=1}^n i^k$ 对 m 取模的值。

- 题解
- 代码实现

Newton 插值法

Newton 插值法是基于高阶差分来插值的方法，优点是支持插入新数据点。

为了实现插入新数据点，我们令：

其中 $\{b_i\}$ 称为 **Newton 基** (Newton basis) 。

若解出 a_i ，则可得到 $p(x)$ 的插值多项式。我们按如下方式定义 **前向差商** (forward divided differences)：

则：

此即 Newton 插值的形式。朴素实现的时间复杂度为 $\mathcal{O}(n^2)$ 。

若样本点是等距的 (即 $x_i = x_0 + i \cdot h$)，令 $t = \frac{x - x_0}{h}$ ，Newton 插值的公式可化为：

上式称为 **Newton 前向差分公式** (Newton forward divided difference formula) 。

▼ Note

若样本点是等距的，我们还可以推出：

其中 $\Delta^k f_0$ 为 **前向差分** (forward differences)，定义如下：

► 代码实现 ([Luogu P4781【模板】拉格朗日插值](#))

横坐标是连续整数的 Newton 插值

例如：求某三次多项式 $p(x)$ 的多项式系数，已知 $p(0)$ 至 $p(3)$ 的值分别为 y_0, y_1, y_2, y_3 。

第一行为 y_0, y_1, y_2, y_3 的连续的前 n 项；之后的每一行为之前一行中对应的相邻两项之差。观察到，如果这样操作的次数足够多 (前提是 $p(x)$ 为多项式)，最终总会返回一个定值。

计算出第 k 阶差分的首项为 $\frac{h^k}{k!} p^{(k)}(x_0)$ ，第 k 阶差分的首项对 $p(x)$ 的贡献为 $\binom{n}{k} \frac{h^k}{k!} p^{(k)}(x_0)$ 次。

时间复杂度为 $\mathcal{O}(n^2)$ 。

C++ 中的实现

自 C++ 20 起，标准库添加了 [std::midpoint](#) 和 [std::lerp](#) 函数，分别用于求中点和线性插值。

习题

- [「NOIP2020」微信步数](#)
- [「联合省选 2022」填树](#)
- [「NOI2019」机器人](#)

参考资料

1. [Interpolation - Wikipedia](#)
 2. [Newton polynomial - Wikipedia](#)
-

本页面最近更新：2025/2/26 23:20:22, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[caibyte](#), [Tiphereth-A](#), [Watersail2005](#), [AtomAlpaca](#), [billchenchina](#), [Chrogeek](#), [Early0v0](#), [EndlessCheng](#), [Enter-tainer](#), [Ghastlcon](#), [Henry-ZHR](#), [hly1204](#), [hsfzLZH1](#), [lr1d](#), [kenlig](#), [Marcythm](#), [megakite](#), [Peanut-Tang](#), [qwqAutomaton](#), [qz-cqy](#), [StudyingFather](#), [swift-zym](#), [swiftqwq](#), [TrisolarisHD](#), [x4Cx58x54](#), [Xeonacid](#), [xiaopangfeiyu](#), [YanWQ-monad](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用